



Plataforma de Gestión de Gastos Personales y Ahorro Estratégico

Documentación Técnica — MVP v1.0

1. Descripción del Proyecto

Ahorra es una plataforma web que permite a los usuarios llevar una bitácora detallada de sus gastos diarios y planificar estratégicamente métodos para mantener sus finanzas controladas. A diferencia de otras apps de finanzas, Ahorra combina el tracking de gastos con un motor de predicción inteligente que ayuda al usuario a visualizar cuándo podrá alcanzar sus metas de ahorro y qué ajustes debe hacer para lograrlo más rápido.

El diferenciador principal es la función de predicción con metas concretas, pensada para el contexto latinoamericano, con soporte para múltiples monedas y recomendaciones generadas por IA (Claude API).

2. Tabla de Casos de Uso

Los siguientes casos de uso representan todas las interacciones principales entre los actores del sistema (usuario y sistema automatizado) y la plataforma Ahorra.

ID	Caso de Uso	Actor	Descripción	Precondición	Resultado Esperado
UC-01	Registro de usuario	Usuario nuevo	El usuario se registra con email/contraseña o Google OAuth	No tener cuenta activa	Cuenta creada y redirige al Home
UC-02	Inicio de sesión	Usuario registrado	El usuario accede con sus credenciales o Google	Cuenta existente	Sesión iniciada, acceso al Home

ID	Caso de Uso	Actor	Descripción	Precondición	Resultado Esperado
UC-03	Registrar sueldo mensual	Usuario autenticado	El usuario ingresa su ingreso mensual base	Sesión activa	Sueldo guardado y visible en Home
UC-04	Agregar categoría de gasto	Usuario autenticado	El usuario crea una categoría (ej: comida, gasolina)	Sesión activa	Categoría disponible para registrar gastos
UC-05	Registrar gasto	Usuario autenticado	El usuario registra un gasto en una categoría con monto y fecha	Al menos una categoría creada	Gasto guardado, ahorro esperado recalculado
UC-06	Ver resumen del mes	Usuario autenticado	El usuario visualiza el dashboard con gastos, sueldo y ahorro esperado del mes actual	Sueldo y gastos registrados	Vista con totales por categoría y ahorro restante
UC-07	Crear meta de ahorro	Usuario autenticado	El usuario define una meta (ej: Viaje a Japón - Q30,000)	Sesión activa	Meta guardada con monto objetivo
UC-08	Ver predicción de ahorro	Usuario autenticado	El usuario ve en cuánto tiempo alcanza su meta dado su ahorro mensual promedio	Al menos una meta y un mes de datos	Fecha estimada y recomendaciones mostradas
UC-09	Configurar gastos recurrentes	Usuario autenticado	El usuario define gastos fijos mensuales por defecto para el cálculo de predicciones	Sesión activa	Gastos recurrentes guardados y usados en predicciones
UC-10	Simular escenario	Usuario autenticado	El usuario ajusta sliders de categorías para ver cómo cambia el tiempo para alcanzar su meta	Meta creada y predicciones activas	Nueva fecha estimada según ajustes
UC-11	Ver historial de meses	Usuario autenticado	El usuario navega entre meses anteriores para ver sus gastos históricos	Al menos 2 meses con datos	Vista de gastos y ahorro de meses pasados

ID	Caso de Uso	Actor	Descripción	Precondición	Resultado Esperado
UC-12	Recibir alerta de presupuesto	Sistema	El sistema notifica al usuario cuando está por exceder el 80% de una categoría	Categoría con límite configurado	Notificación visible en la app
UC-13	Dividir capital en metas	Usuario autenticado	El usuario distribuye su ahorro actual entre múltiples metas (ej: 20K viaje, 10K carro)	Capital disponible registrado	Distribución guardada y proyección por meta
UC-14	Editar perfil de cuenta	Usuario autenticado	El usuario actualiza nombre, email o contraseña	Sesión activa	Datos actualizados en la cuenta
UC-15	Cerrar sesión	Usuario autenticado	El usuario cierra su sesión activa	Sesión activa	Sesión terminada, redirige al Login

3. Pantallas del Sistema

A continuación se detallan todas las pantallas necesarias para el MVP, con su ruta sugerida dentro de la aplicación Next.js y los componentes principales que cada una debe contener.

Pantalla	Ruta	Descripción	Componentes Principales
Login	/login	Inicio de sesión con email/contraseña o Google OAuth	Formulario login, botón Google, link a Sign Up
Sign Up	/signup	Registro de nueva cuenta con email o Google	Formulario registro, botón Google, link a Login
Home / Dashboard	/home	Vista principal del mes actual: sueldo, gastos por categoría y ahorro esperado	Card de sueldo, cards de categorías, barra de ahorro, botón agregar gasto

Pantalla	Ruta	Descripción	Componentes Principales
Agregar Gasto	/home/add (modal)	Modal o drawer para registrar un nuevo gasto en una categoría	Selector de categoría, campo de monto, campo de fecha, campo de nota
Detalle de Categoría	/categoria/:id	Vista de todos los gastos dentro de una categoría en el mes actual	Lista de gastos, total gastado, límite configurado, gráfica de evolución
Historial	/historial	Comparativa de meses anteriores con gastos y ahorros	Selector de mes/año, tabla o gráfica de evolución, resumen por categoría
Predicciones	/predicciones	Proyección futura del capital según gastos recurrentes y metas definidas	Timeline de metas, sliders de ajuste por categoría, fecha estimada, recomendaciones
Crear / Editar Meta	/predicciones/meta (modal)	Modal para crear o editar una meta de ahorro con nombre y monto objetivo	Campo de nombre, campo de monto, opción de prioridad
Ajustes de Cuenta	/ajustes	Configuración del perfil, contraseña y preferencias generales	Formulario de perfil, cambio de contraseña, configuración de moneda
Onboarding	/onboarding	Flujo de bienvenida para nuevos usuarios: ingresar sueldo y primeras categorías	Pasos guiados: sueldo → categorías → primera meta

4. Stack Tecnológico

El stack fue seleccionado considerando que el proyecto es desarrollado por un solo programador que necesita moverse rápido, minimizar la complejidad de infraestructura y tener la flexibilidad de integrar IA de forma nativa.

Capa	Tecnología	Versión	Justificación
Frontend	Next.js	14+	Framework React con App Router. SSR para SEO, API Routes integradas, deploy nativo en Vercel. Elimina la necesidad de un servidor separado en el MVP.
UI / Estilos	Tailwind CSS	3+	Utility-first CSS. Desarrollo rápido sin escribir CSS custom. Ideal para un solo dev que necesita velocidad.
Componentes UI	shadcn/ui	Latest	Componentes accesibles y personalizables contruidos sobre Radix UI. Sin dependencia de librería externa pesada.
Gráficas	Recharts	2+	Librería de gráficas para React, ligera y con buena integración. Ideal para dashboards financieros.
Backend / API	Next.js API Routes	14+	Al usar Next.js, el backend vive en el mismo repo. Reduce complejidad de infraestructura para el MVP.
Base de Datos	PostgreSQL (via Supabase)	Latest	Base de datos relacional, ideal para datos financieros estructurados. Supabase provee hosting gratuito y cliente JS.
ORM	Prisma	5+	ORM type-safe para Node/TypeScript. Facilita queries complejas y migraciones de schema.
Autenticación	Supabase Auth	Latest	Incluye Google OAuth out-of-the-box. Elimina semanas de implementar auth desde cero.
Lenguaje	TypeScript	5+	Tipado estático que reduce bugs en datos financieros donde los tipos importan mucho.
IA / Asistente	Claude API (claude-sonnet-4)	Latest	Para las recomendaciones inteligentes de ahorro y análisis de patrones de gasto. Se integra via API REST.
Dev Assistant	Claude Code	Latest	Asistente de desarrollo en terminal. Acelera la generación de componentes, migrations y lógica de negocio.
Deploy Frontend	Vercel	Latest	Deploy automático desde GitHub. Tier gratuito suficiente para MVP. Integración nativa con Next.js.
Deploy DB / Auth	Supabase	Latest	Tier gratuito incluye 500MB DB, auth y 2GB de ancho de banda. Suficiente para validar el producto.

Capa	Tecnología	Versión	Justificación
Control de versiones	GitHub	Latest	Repositorio único (monorepo Next.js). CI/CD automático con Vercel al hacer push a main.

5. Estrategia de Despliegue

5.1 Infraestructura

La arquitectura de despliegue está diseñada para ser completamente gratuita durante la fase de MVP y validación. No se utilizará Docker en esta etapa para reducir la complejidad operativa.

Servicio	Proveedor	Costo inicial	Límites del tier gratuito
Frontend / API Routes	Vercel	Gratis	100GB bandwidth, builds ilimitados, dominios custom
Base de Datos	Supabase	Gratis	500MB storage, 2GB bandwidth, 50,000 usuarios auth/mes
Autenticación	Supabase Auth	Gratis	Incluido en Supabase, Google OAuth incluido
Dominio	Namecheap / GoDaddy	~\$12/año	Dominio .com o .app personalizado
Claude API (IA)	Anthropic	Pay-per-use	Sin tier gratuito, costo muy bajo por llamada (~\$0.003/1K tokens)

5.2 Flujo de CI/CD

1. El desarrollador hace push a la rama main en GitHub.
2. Vercel detecta el push automáticamente y ejecuta el build de Next.js.
3. Si el build es exitoso, Vercel despliega la nueva versión en producción en ~30 segundos.
4. Las variables de entorno (Supabase URL, Claude API Key) se gestionan en el panel de Vercel.

6. Arquitectura Sugerida

6.1 Visión General

Se propone una arquitectura monorepo con Next.js como núcleo central. Este enfoque elimina la necesidad de gestionar dos repositorios separados y simplifica enormemente el desarrollo en solitario.

6.2 Estructura del Repositorio

ahorra/

```
├─ app/                # Next.js App Router
│   ├─ (auth)/         # Grupo de rutas: login, signup, onboarding
│   ├─ (dashboard)/    # Grupo de rutas protegidas: home, historial, predicciones
│   ├─ ajustes/        # Configuración de cuenta
│   └─ api/            # API Routes (backend serverless)
│       ├─ gastos/     # CRUD de gastos
│       ├─ categorias/ # CRUD de categorías
│       ├─ metas/      # CRUD de metas de ahorro
│       └─ ai/         # Endpoint para llamadas a Claude API
├─ components/        # Componentes React reutilizables
│   ├─ ui/            # shadcn/ui components
│   ├─ charts/        # Componentes de Recharts
│   └─ forms/         # Formularios tipados
├─ lib/               # Utilidades y helpers
│   ├─ supabase.ts    # Cliente de Supabase
│   ├─ prisma.ts      # Cliente de Prisma
│   └─ claude.ts      # Helper para Claude API
├─ prisma/            # Schema y migraciones de base de datos
│   └─ schema.prisma
└─ types/             # Tipos TypeScript globales
```

6.3 Modelo de Base de Datos (Entidades Principales)

Las entidades principales del sistema y sus relaciones son:

Entidad	Campos Principales	Relaciones
users	id, email, name, avatar_url, created_at	1:N con meses, metas, categorías
months	id, user_id, year, month, salary, created_at	1:N con gastos; N:1 con users
categories	id, user_id, name, icon, color, monthly_limit	1:N con gastos; N:1 con users
expenses	id, month_id, category_id, amount, note, date	N:1 con months y categories
goals	id, user_id, name, target_amount, current_amount, deadline	N:1 con users
recurring_expenses	id, user_id, category_id, amount, description	N:1 con users y categories

6.4 Integración con Claude API

La IA de Anthropic se integra en dos puntos clave de la aplicación:

Funcionalidad	Prompt / Uso	Cuándo se dispara
Recomendaciones de ahorro	Recibe gastos del mes y metas del usuario, genera 3 recomendaciones concretas para reducir gastos y llegar antes a las metas.	Al cargar la pantalla de Predicciones
Análisis de patrones	Analiza historial de meses pasados e identifica categorías donde el usuario gasta consistentemente más de lo presupuestado.	Al ver el Historial con 3+ meses de datos
Meta inteligente	Dado un monto objetivo y el ahorro mensual promedio, calcula el tiempo estimado y sugiere un colchón de seguridad adicional.	Al crear una nueva meta de ahorro

6.5 Claude Code y Templates

Para acelerar el desarrollo, se propone el uso de Claude Code junto con templates predefinidos que el asistente puede generar o completar automáticamente:

Template	Descripción	Comando sugerido con Claude Code
API Route CRUD	Template de Next.js API Route con validación Zod y manejo de errores estándar	claude 'genera un API route para la entidad [X] con Prisma y Zod'
Componente Dashboard Card	Card reutilizable para mostrar métricas financieras con título, valor y tendencia	claude 'crea un componente MetricCard con shadcn/ui'
Prisma Migration	Schema de Prisma para una nueva entidad con relaciones y tipos correctos	claude 'agrega la entidad [X] al schema de Prisma con relación a users'
Página protegida	Template de página Next.js con verificación de sesión Supabase y layout de dashboard	claude 'crea la página /predicciones con auth check de Supabase'
Hook de datos	Custom React hook con SWR o React Query para fetching de datos tipados	claude 'crea un hook useGastos que llame a /api/gastos con SWR'

7. Roadmap de Desarrollo (Solo Dev)

Fase	Duración	Entregables
Fase 1 — Base	Semanas 1-3	Auth con Supabase + Google OAuth, onboarding, registro de sueldo y categorías, CRUD de gastos, dashboard del mes actual

Fase	Duración	Entregables
Fase 2 — Core completo	Semanas 4-6	Historial de meses, límites por categoría, alertas de presupuesto, gráficas con Recharts
Fase 3 — Diferenciador	Semanas 7-9	Motor de predicciones, metas de ahorro, simulador de escenarios, integración Claude API para recomendaciones
Fase 4 — Lanzamiento	Semanas 10-12	Onboarding pulido, landing page, responsive móvil, ajustes de UX, dominio custom y deploy en Vercel